Technical University of Crete
School of Electrical and Computer Engineering
Course: Reinforcement Learning and Dynamic Optimization
Stock Portfolio Optimization
Phase 2: Stock Market Trading

Delivery Date: July 21, 2024
Instructor: Thrasyvoulos Spyropoulos

	Last Name	**Name**	**AM**
1	Antonogiannakis	Ioannis	2020030018
2	Paraskevopoulou	Despoina	2021030135

Stock Portfolio Problem

In this phase of the project, we assume we are in an unknown environment, where it is necessary to learn while optimizing. First we implemented the tabular Q-Learning algorithm for small to medium scale scenarios, while, later, for larger versions of the problem we used the DQN algorithm with Deep Neural Networks as approximators. Finally we compared the results of the two algorithms with each other, as well as with the results of Policy Iteration (PI), which we assume from Phase 1 to be optimal.

Task 1

In the first task, we assume that we do not know the parameters of the Markov chain driving the price of each stock. We know that each stock has two states H, L but we do not know the stock rewards r_H^i, r_L^i nor the transition probabilities P_{xx}^i where $x \in \{H, L\}$ and $i \in \{0, 1, ..., N-1\}$. For this version of the problem, we implemented tabular Q-learning for several scenarios, ensuring the optimality of the algorithm's performance.

As ground truth, we consider the policy extracted from the Policy Iteration implementation we used for Phase 1.

We started with some "toy" scenarios and later we continued with larger, non trivial ones.

- The first scenario was for $N = 2$ stocks, $\gamma = 0$, $r_1^H = 2r_2^H$ and the same r_i^L, $fee = 0.1$, $LR = 0.001$.

  We chose those parameters so we can have a scenario that the optimal policy is obvious, which in this case would be to always stick with the same starting stock, due to the relatively high fee and the "impatient" γ.

  We run one "long" episode (20.000 steps).

  Implementing the above scenario, we observed that Q-learning chose the optimal policy, matching the one that PI indicates.

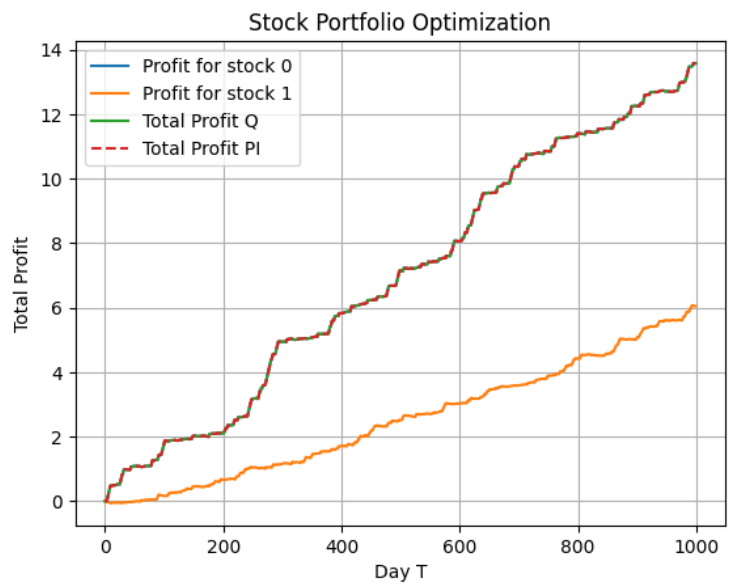


Figure 1: Cumulative Profits

In the plot above we see the cumulative profits resulting from PI (dashed red lines) and Q-learning (continuous green line) over a time horizon of $T = 1000$ days. We observe that the profits are the same, proving that Q-learning works optimally for this scenario, by consistently choosing stock 0. We also implemented the scenario where the stock we chose first was stock 1 and, again, Q-learning stuck with stock 1 for the entire horizon.

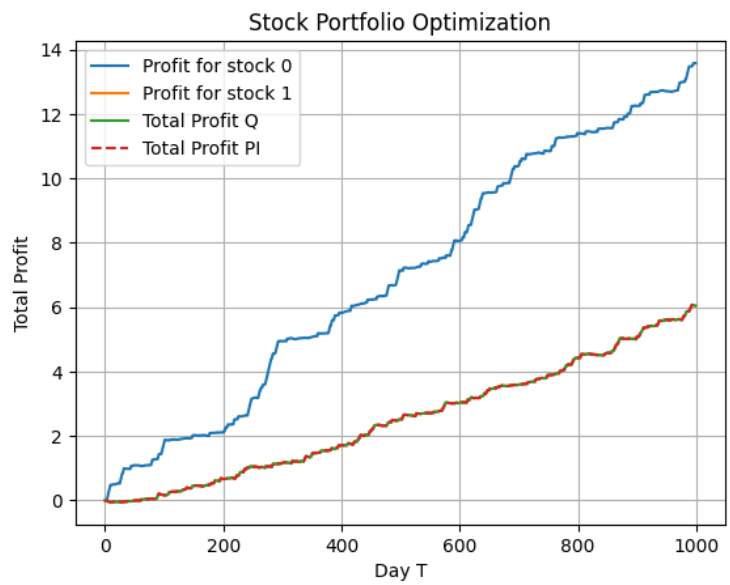


Figure 2: Cumulative Profits

- In the second "toy" scenario, we run for N=2, $\gamma = 0.9$, $p^i_{HH} = 0.4$, $p^i_{LH} = 0.1$, $fee = 0.01$, $LR = 0.001$ for episode length= 50.000. Again, the policies were overlapping, proving that, again, in this scenario Q-learning works optimally.

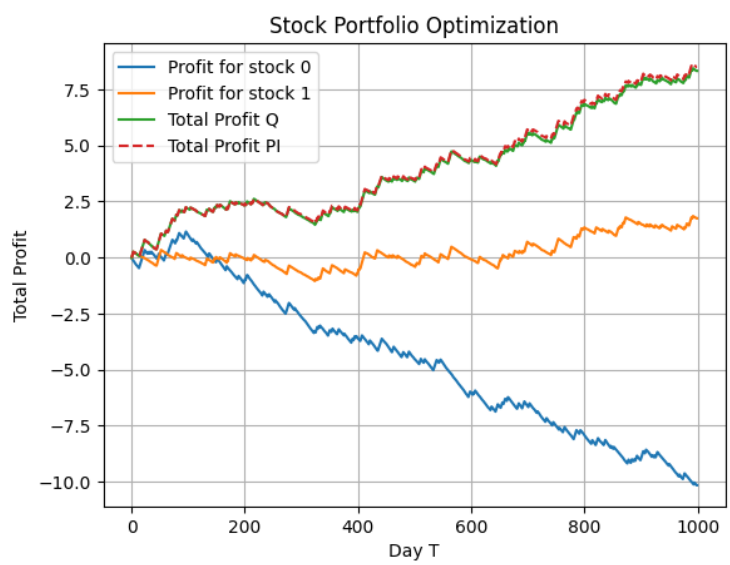


Figure 3: Cumulative Profits for second scenario

Above we see the cumulative profits extracted from the two algorithms with respect to the time horizon. We observe that the policies are "acting" adequately with the

changes of the stocks (blue and orange lines).

Task 2

For the second task, we implemented larger and non trivial scenarios in order to ensure the optimality of Q-learning.

- As it was mentioned in Phase 1, the largest scenario we could run with Policy Iteration was for $N = 9$ stocks. So we implemented this scenario using $\gamma = 0.90$, $p^i_{HH} = 0.4$, , $p^i_{LH} = 0.1$, $fee = 0.01$, $LR = 0.001$. The values of the probabilities were chosen aiming to simulate realistic phase changes of real stocks. The output of this scenario is shown below:

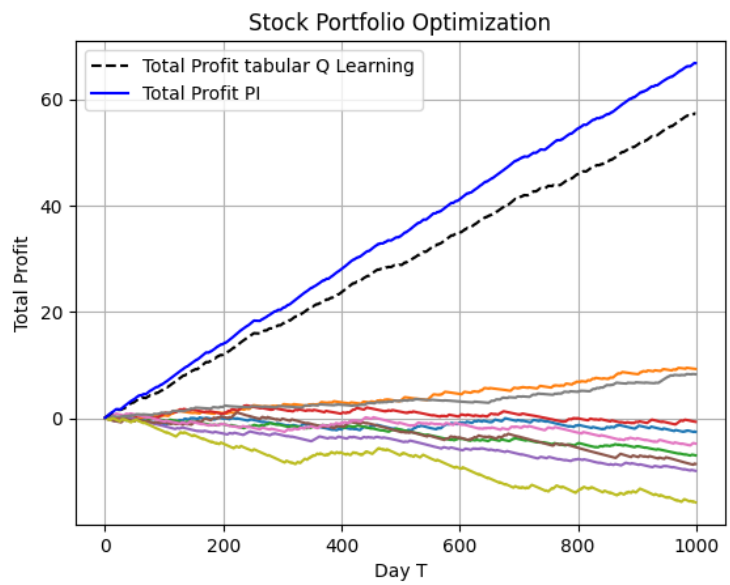


Figure 4: Cumulative Profits for N=9 stocks

  We observe that the two algorithms have pretty similar performance. Q-learning (black dashed line) approximates pretty well the optimal policy of PI (blue line) in the time horizon. We run this scenario for episode length = 3.000.000, sufficient time for it to learn and converge.

- We also run a scenario for $N = 5$ stocks for $\gamma = 0.1$, $p^i_{HH} = 0.4$, , $p^i_{LH} = 0.1$, $fee = 0.01$, $LR = 0.001$ and we compared it with a scenario for the same stocks but with $\gamma = 0.9$.

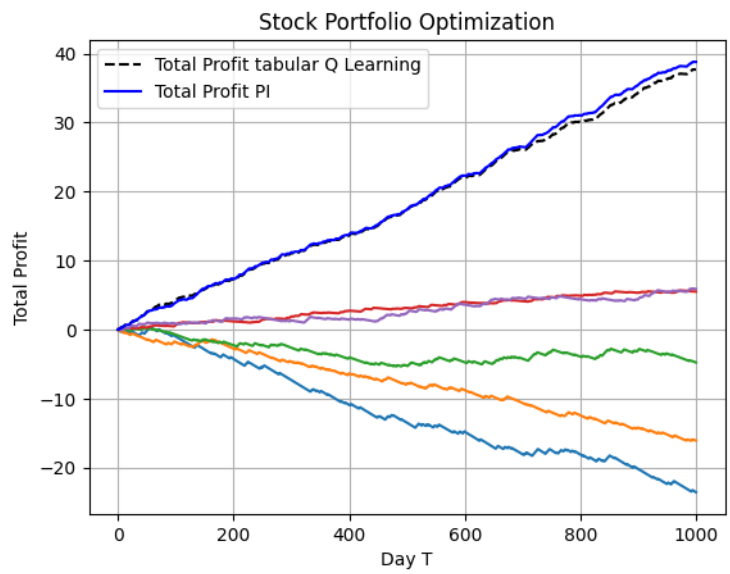


Figure 5: Cumulative Profits for N=5 stocks, $\gamma = 0.1$

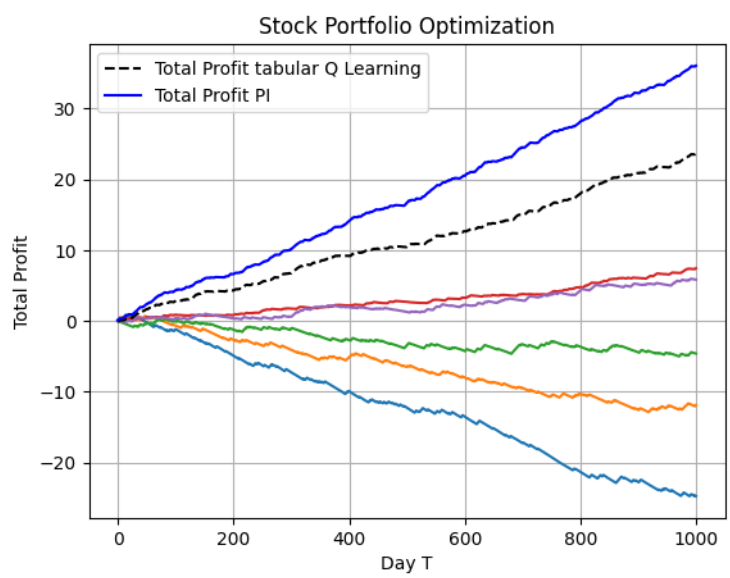


Figure 6: Cumulative Profits for N=5 stocks, $\gamma = 0.9$

We observe that the scenario with $\gamma = 0.1$ leads to faster convergence of the Q-learning and better resulting policy. This happens because for low γ the algorithm focuses on short-term rewards, which might be favorable in this scenario, leading to rapid learning. The scenario with higher γ would require more iterations to converge to the optimal policy.

Hyperparameter Tuning

- Learning rate: Theoretically, we need a learning rate that drops neither too quickly nor too slowly, satisfying the following relations:

$$\sum_{t=1}^{\infty} LR_t = \infty$$

$$\sum_{t=1}^{\infty} LR_t^2 < \infty$$

  In practice, we found out that a "small" and constant LR (0.001, 0.002), works really well.

- Epsilon: Similar conditions with the learning rate should be satisfied for epsilon as well. So, we used $epsilon = \frac{1}{i}$ where i is the episode step number.

Task 3

In task 3, we implemented a DQN algorithm for medium and large scenarios as in task 2. Specifically, we implement DQN with experience replay, using a buffer of size 10.000. The Neural Network we constructed was of three linear layers (one input, one output and one hidden). The price change of each stock was still Markov chain based.
As activation function we used ReLU, optimizer was SGD and loss was computed with the MSE formula.
First, we implemented DQN knowing the phases H(igh), L(ow) of each stock. This is basically a "test" implementation, in order to check that the DQN algorithm performs at least just as well as tabular Q-learning, when the Neural Network gets plenty of information about the environment.
The input we used for the Neural Network in this initial implementation, was a N+1 sized vector where the first element is the current stock we have invested in and the rest are the phases of the N stocks.
The output was a vector of size N, containing Q values, representing the possible actions. Basically, the highest Q value gives us the "best" stock.
As learning rate we have $LR = 0.001$, $\gamma = 0.9$, batch size= 16, and ϵ dropped with rate $10^{-2} * episode$, because we want to decrease the probability of exploring at each episode.
Again, we used PI performance as ground truth, considering the agreement with that sufficient. We run an experiment of $N = 5$ stocks for PI, Q-learning and DQN with phase knowledge in order to check the optimality of the DQN implementation.

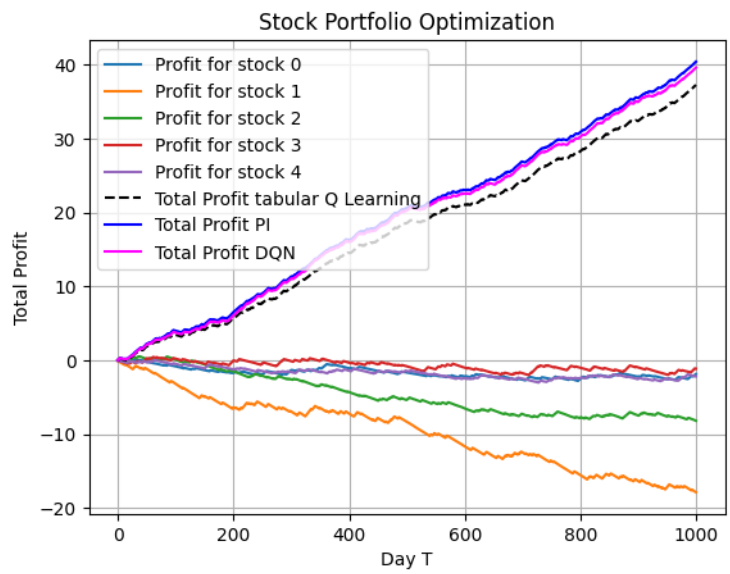


Figure 7: Cumulative Profits for DQN with phase knowledge

Above we see the cumulative profit form the DQN implementation with phase knowledge. As we see the profits of DQN (magenta line) are in sync with the optimal output of the PI, plotted for the time horizon T. We, also observed that tabular Q -learning converges slower to the optimal policy than the DQN algorithm (1 episode of 200.000 steps for the Q-learning vs 100 episodes of length 250 for DQN). That is because of the generalization and approximation performed by the Neural Network.

Having this as an optimal implementation, we proceeded to the scenario where our agent doesn't know the phases H, L of each stock, but rather sees continuous values, representing the yield of each stock. We constructed a Neural Network which takes as input the yields of the stocks at each day plus the yield of the stock we picked, so a N+1 size vector. The output is, again, the N size vector that contains the Q values, representing the possible actions. For this environment we run the following scenarios:

- First, we run a scenario for $N = 4$ stocks, $fee = 0.01$, learning rate for both Q-learning and DQN $LR = 0.001$ and $\gamma = 0.9$. For DQN we chose 100 episodes of episode length= 200 each. The output of this experiment was the following:

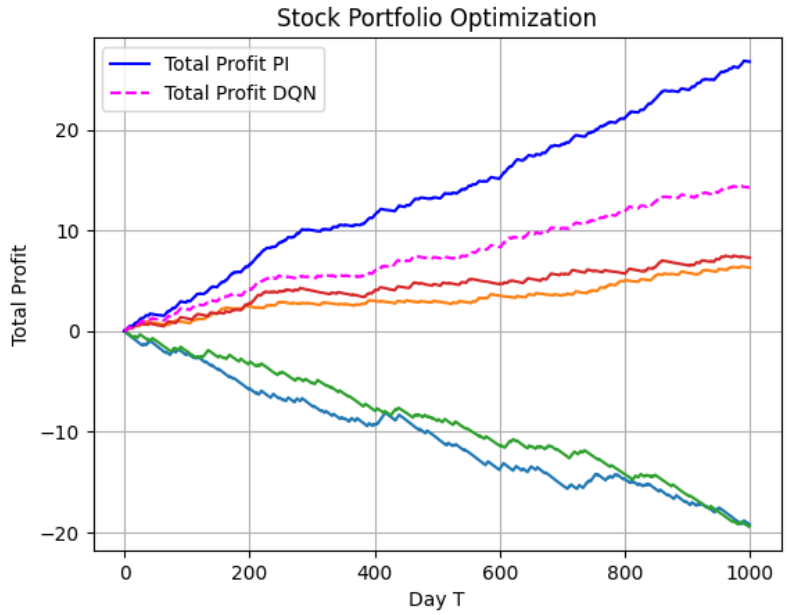


Figure 8: Cumulative Profits for DQN with no phase knowledge

- Secondly, we tried a larger scenario, we run DQN for $N = 8$ stocks, changing the number of episodes to 200 and their length to 2.000 and we got the following results:

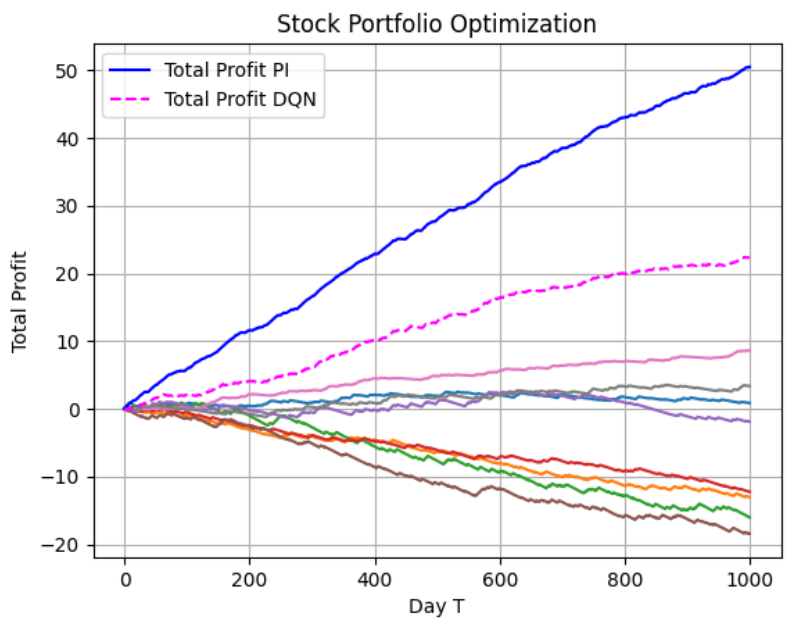
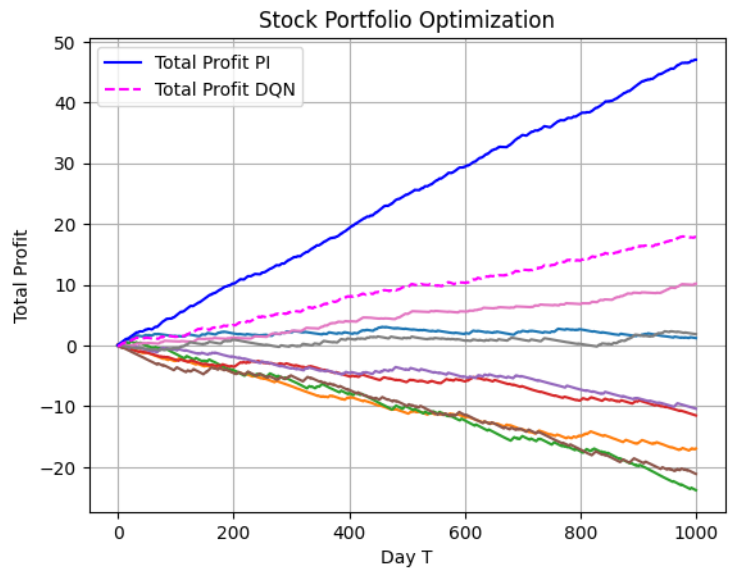


Figure 9: Cumulative Profits for DQN with no phase knowledge, N=8

We observe that the algorithm has suboptimal performance, but it still learns. The DQN algorithm with no phase knowledge required bigger episode length and more episodes than the DQN with phase knowledge.

- Additionally, we run an episode for the same scenario as above but with $\gamma = 0.2$. The results were the following:



We notice that the scenario with $\gamma = 0.9$ has a slightly better performance probably because this implementation favored far sighted strategies.

Conclusion Overall, both algorithms performed optimally when they run with phase

knowledge. DQN was faster than Q-learning, needing less iterations to converge to the optimal policy suggested by Policy Iteration. Lastly, when there was no knowledge of the phases of the stocks, DQN performed suboptimally, but still managed to learn a viable strategy and accumulate some profits.